Task Manager Architecture and Component Design

This document describes the application's architecture, component tree, module dependencies, data model, main flows, workflow permissions, and possible improvements. Paths and function names refer to the source files in this repository.

Item	Implementation
Project	`task-manager/` , using the Next.js App Router
Stack	Next.js **15.5.28**, React **19.1.0**, TypeScript 5 in strict mode, Tailwind CSS 4
Runtime	A single Node.js process; no middleware, external database, or third-party authentication library
Storage	Local JSON file at `data/db.json` ; override with `DB_PATH`
Session	HMAC-SHA256 signed payload in the `tm_session` httpOnly cookie, valid for 8 hours
Passwords	Node's built-in `scrypt` , stored as `salt:hash` and compared with `timingSafeEqual`
Tests	`node:test` with `tsx` : 13 cases, comprising 8 workflow tests and 5 storage tests

0. Overview

```
Browser (React 19)
  |-- Navigation -> Next.js RSC pages -> lib/store.ts -> data/db.json
  `-- Form POST  -> Server Actions -> domain validation -> storage writes
```

Five design principles organize the application:

1. **Server Components handle rendering; Client Components handle interaction.** Every page is a Server Component. Only `LoginForm` , `NewTaskForm` , and `SubmitButton` use `"use client"` .
2. **Server Actions own mutations.** All five actions live in `lib/actions.ts` ; pages render the forms that call them.
3. **One transition table defines the workflow.** The detail page, server validation, and tests all use `TRANSITIONS` in `lib/workflow.ts` .
4. **Permission checks serve two purposes.** Pages hide unavailable actions for usability; Server Actions enforce authorization.
5. **Storage has a narrow interface.** `lib/store.ts` is the only application module that accesses the filesystem. Tests use the filesystem separately to create temporary databases.

1. Overall architecture

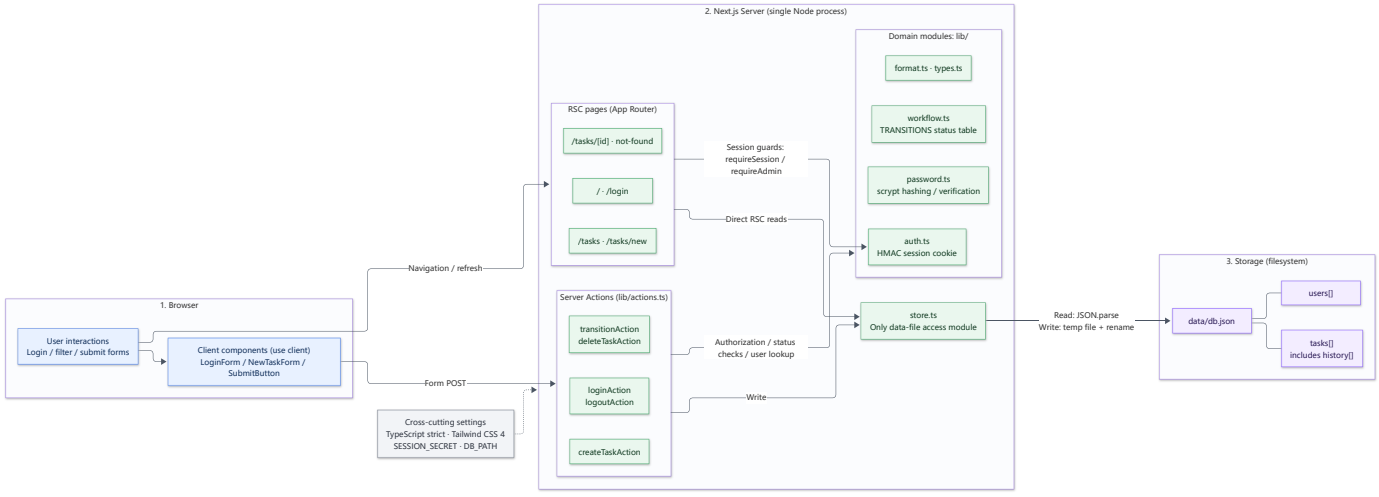


Figure 1. Browser, Next.js server, and JSON storage, with separate read and write paths.

1.1 Responsibilities by layer

The diagram groups code into three runtime domains. The table below divides the same code into five responsibilities.

Layer	Files or components	Responsibility	Boundary
Browser	HTML/RSC payload and three Client Components	Display, input, pending states, confirmation dialogs	Forms call Server Actions; no independent authorization logic or custom fetch API
Pages (RSC)	`app/layout.tsx`, `app/page.tsx`, `app/login/page.tsx`, `app/tasks/page.tsx`, `app/tasks/new/page.tsx`, `app/tasks/[id]/page.tsx`, `app/not-found.tsx`	Read the session, load and filter data, render available actions	Reads only; delegate mutations to actions
Server Actions	`lib/actions.ts`	Parse input, authenticate, authorize, validate, mutate, revalidate, redirect	All three task actions check the session; failed validation must not write data
Domain modules	`lib/workflow.ts`, `lib/auth.ts`, `lib/password.ts`, `lib/format.ts`, `lib/types.ts`	Workflow rules, session signatures, password hashing, formatting, types	Keep workflow and formatting logic independent of storage; authentication also uses Next.js cookies and redirects
Storage	`lib/store.ts`	Read/write JSON, seed data, generate IDs, replace the database file atomically	The only application module that imports `node:fs`

1.2 Runtime and deployment

Aspect	Current behavior
Process model	One Next.js Node process: `next dev` for development or `next build` followed by `next start` for production
External services	No external database, authentication provider, or cache
Persistence	`data/db.json` is initialized when the store is first read; tests set `DB_PATH` to a temporary file
Middleware	No `middleware.ts`; pages and actions perform permission checks
Intended use	A single instance for a small demonstration or review; concurrent writes and multiple replicas require storage changes

1.3 Read and write paths

Aspect	Read path	Write path
Entry point	Direct calls from RSC pages	Client form or page form calling a Server Action
Call chain	`page.tsx` -> `listTasks()` / `getTask()` -> `readDb()`	`actions.ts` -> validation -> `createTask()` / `transitionTask()` / `deleteTask()` -> `writeDb()`
HTTP API	No separate REST API	Server Actions use Next.js transport and remain public POST endpoints that require server-side validation
Failure	`notFound()` or redirect to login	Inline `{ error }` response or redirect with an error code
Freshness	The task list and detail pages explicitly use `dynamic = "force-dynamic"`; other session-aware pages become dynamic through `cookies()`	Mutations call `revalidatePath()` and redirect

There is no standalone API layer. Mobile clients and third-party integrations would require a new public interface.

2. Pages, components, and actions

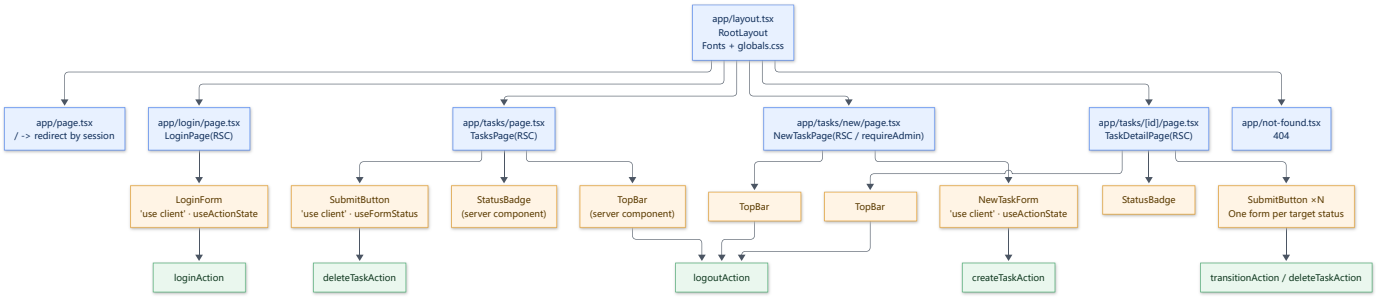


Figure 2. Pages compose components; forms call Server Actions; pages read data through library modules.

2.1 Pages

Route	File	Guard	Data	Interaction
`/`	`app/page.tsx`	Read session, then redirect	`getSession()`	None
`/login`	`app/login/page.tsx`	Redirect authenticated users to `/tasks`	`getSession()`	`LoginForm`
`/tasks`	`app/tasks/page.tsx`	`requireSession()`	`listTasks()`, status query filter, per-status counts	Status tabs; administrators also see create and delete actions
`/tasks/new`	`app/tasks/new/page.tsx`	`requireAdmin()`	No task read	`NewTaskForm`
`/tasks/[id]`	`app/tasks/[id]/page.tsx`	`requireSession()`	`getTask(id)`, `nextStatuses(status)`	History; administrators also see transition and delete forms
404	`app/not-found.tsx`	None	None	Return to the task list

2.2 Components

Component	File	Type	Props	Dependencies and responsibility
`LoginForm`	`components/LoginForm.tsx`	Client	None	`loginAction`; controlled inputs, `useActionState`, inline errors, demo-account autofill
`NewTaskForm`	`components/NewTaskForm.tsx`	Client	None	`createTaskAction`; title, description, assignee, errors, disabled pending state
`SubmitButton`	`components/SubmitButton.tsx`	Client	`children`, `pendingLabel`, `variant`, `className`, `confirmText`	`useFormStatus`; shared styling, pending text, optional native confirmation
`StatusBadge`	`components/StatusBadge.tsx`	Server	`status`	`STATUS_META`; status label, dot, and colors
`TopBar`	`components/TopBar.tsx`	Server	`session`	`logoutAction`; product title, current user and role, sign out

The three Client Components handle input and submission. They do not read the database or enforce permissions.

2.3 Action call matrix

Action	Caller	Authentication and authorization	Side effects
`loginAction`	`LoginForm`	Validate email and password	Set session cookie, redirect to `/tasks`
`logoutAction`	`TopBar`	No session required to delete the cookie	Delete cookie, redirect to `/login`

Action	Caller	Authentication and authorization	Side effects
`createTaskAction`	`NewTaskForm`	Session and Administrator role	Create a Pending task, revalidate, redirect to detail
`transitionAction`	Detail-page transition forms	Session, Administrator role, valid transition	Append history, update status, revalidate, redirect with `?ok=1`
`deleteTaskAction`	List and detail delete forms	Session and Administrator role	Remove task, revalidate, redirect to `/tasks?ok=deleted`

Task actions share the same pattern: session, role, business validation, persistence, then revalidation. New task actions should follow that sequence.

3. Library modules and dependency direction

Module	Main exports	Consumers	Purpose
`lib/types.ts`	`Role`, `TaskStatus`, `TASK_STATUSES`, `Task`, `TaskEvent`, `User`, `PublicUser`	Most modules	Types and constants; no application dependencies
`lib/workflow.ts`	`TRANSITIONS`, `nextStatuses`, `canTransition`, `isTerminal`, `isTaskStatus`, `STATUS_META`	Pages, actions, tests	Workflow rules and presentation metadata
`lib/password.ts`	`hashPassword`, `verifyPassword`	Seed storage, login action	scrypt and constant-time comparison
`lib/auth.ts`	`SESSION_COOKIE`, `Session`, `createToken`, `verifyToken`, `getSession`, `requireSession`, `requireAdmin`, `startSession`, `endSession`	Pages, actions	HMAC sessions, cookies, page guards
`lib/store.ts`	`listTasks`, `getTask`, `findUserByEmail`, `createTask`, `transitionTask`, `deleteTask`	Pages, actions	Filesystem access, seed data, atomic file replacement
`lib/actions.ts`	Five actions and `FormState`	Forms, pages	Server-side mutation entry points
`lib/format.ts`	`formatDateTime`, `formatDate`, `relativeTime`, `initials`	Pages, `TopBar`	Time and avatar formatting; dates use `Asia/Shanghai`

```
Pages / components -> actions -> auth / password / workflow / store
Pages              -> auth / store / workflow / format
Library modules    -> shared types and constants
```

Review these boundaries:

1. Pages and components must not import `node:fs` or write `data/db.json` directly.
2. Client Components must not import `lib/store.ts`, which depends on Node's filesystem. They can import actions and browser-compatible helpers.
3. Status labels and colors belong in `STATUS_META`, rather than being copied into pages.

4. Data model

TASKS			
string	id	PK	T-001 (maximum number + 1)
string	title		Action validates at least 4 characters
string	description		Action validates at least 10 characters
string	assignee		Free text; defaults to Unassigned
string	status		Pending | Approve | To Do | Done | Cancelled
string	createdBy		Name string, not a foreign key
string	createdAt		ISO 8601
string	updatedAt		ISO 8601; list sorted descending
array	history		TaskEvent[]; append on transition

USERS			
string	id	PK	u_admin / u_viewer
string	name		Display name
string	email	UK	Case-insensitive login
string	passwordHash		scrypt: salt:hash
string	role		Administrator | Viewer

history[] (embedded array, not a table)

TASK_EVENTS		
string	at	ISO timestamp
string	from	Source status; null on creation
string	to	Target status
string	by	Actor name
string	note	Note (seed values only; empty at runtime)

Figure 3. Users, tasks, and embedded task history. Assignee and creator fields store names rather than foreign keys.

4.1 Type definitions

```
export type Role = "Administrator" | "Viewer";
export const TASK_STATUSES = ["Pending", "Approve", "To Do", "Done", "Cancelled"] as const;
export type TaskStatus = (typeof TASK_STATUSES)[number];
export type TaskEvent = {
  at: string; from: TaskStatus | null; to: TaskStatus; by: string; note: string;
};
export type Task = {
  id: string; title: string; description: string; assignee: string;
  status: TaskStatus; createdBy: string; createdAt: string; updatedAt: string;
  history: TaskEvent[];
};
export type User = {
  id: string; name: string; email: string; passwordHash: string; role: Role;
};
export type PublicUser = Pick<User, "id" | "name" | "email" | "role">;
```

4.2 Database file example

```
{
  "users": [
    { "id": "u_admin", "name": "Yuheng Yang", "email": "admin@example.com", "passwordHash": "salt:hash", "role": "Administrator" },
    { "id": "u_viewer", "name": "Yuheng Yang", "email": "viewer@example.com", "passwordHash": "salt:hash", "role": "Viewer" }
  ],
  "tasks": [
    {
      "id": "T-004",
      "title": "Status transition guard",
      "description": "Keep Pending, Approve, To Do and Done in one table shared by the UI and the server.",
      "assignee": "Yuheng Yang",
      "status": "Pending",
      "createdBy": "Yuheng Yang",
      "createdAt": "2026-10-24T06:02:00.000Z",
      "updatedAt": "2026-10-24T06:02:00.000Z",
      "history": [
        { "at": "2026-10-24T06:02:00.000Z", "from": null, "to": "Pending", "by": "Yuheng Yang", "note": "Created" }
      ]
    }
  ]
}
```

This example shows one task. The seed contains two users and six tasks, `T-001` through `T-006`, covering all five statuses. Seed timestamps are generated relative to the current date by `at(daysAgo, hour, minute)`; example timestamps are illustrative. The ignored local database may contain additional tasks created during use.

4.3 Storage implementation

Concern	Implementation	Location
File path	`process.env.DB_PATH ?? path.join(process.cwd(), "data", "db.json")`	`DB_PATH`
Initialization	A failed read or parse creates and writes two users and six tasks	`readDb()`, `seed()`
Corruption	Any read/parse failure currently triggers reseeding, not just a missing file	`readDb()`
Atomic replacement	Write to `${DB_PATH}.tmp`, then rename over the database	`writeDb()`
ID generation	Maximum existing numeric task ID plus one, padded to three digits	`nextTaskId()`
History	Append an event, update status/timestamp, and persist	`transitionTask()`
Ordering	Sort by `updatedAt` descending in memory	`listTasks()`
Queries	Read all records and filter in memory; no pagination, search, or index	Store and pages
Test isolation	Set `DB_PATH` to a temporary directory before importing the store	`test/store.test.ts`

`assignee`, `createdBy`, and `history.by` contain name strings, not user IDs. Rendering needs no join, but changing an account name does not automatically update those stored strings.

5. Main flows

5.1 Login and sessions

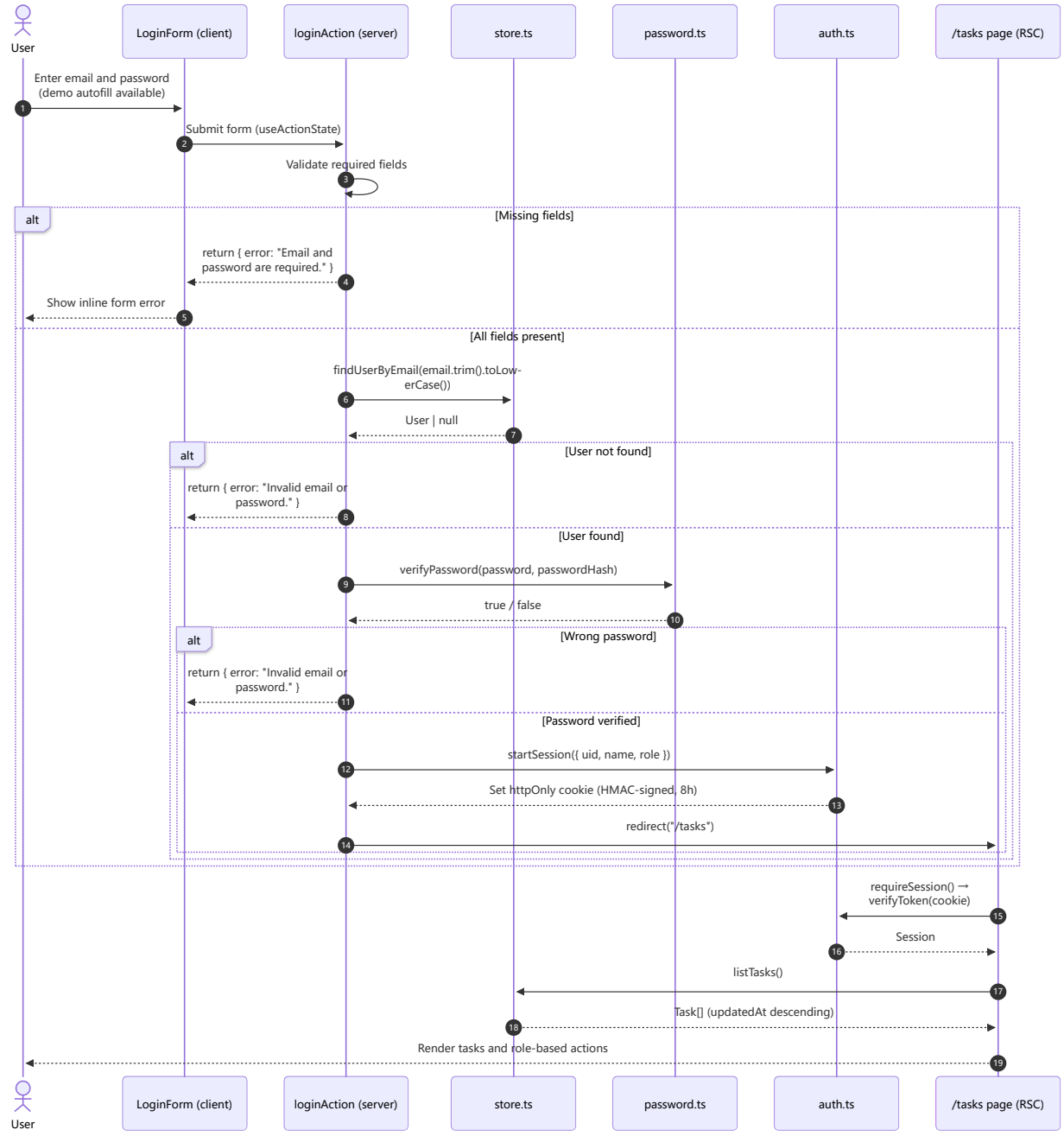


Figure 4. Form submission, user lookup, password verification, signed cookie, and redirect.

Step	Location	Behavior
1. Submit	`LoginForm`	`useActionState(loginAction)` displays returned errors inline
2. Required fields	`loginAction`	Missing email/password returns `{ error }` before a database lookup
3. Find user	`findUserByEmail()`	Trim and lowercase the email before comparison
4. Verify password	`verifyPassword()`	Only called for an existing user; unknown accounts and wrong passwords receive the same error
5. Start session	`startSession()`	Set httpOnly, SameSite=Lax, eight-hour cookie; add Secure in production
6. Redirect	`redirect("/tasks")`	An already authenticated visitor to `/login` also goes to the task list

Step	Location	Behavior
7. Guard pages	`requireSession()`, `requireAdmin()`	Redirect visitors without the required session or role; actions check `getSession()` independently

Missing fields return `Email and password are required.` Invalid credentials return `Invalid email or password.` Unauthenticated access to protected pages redirects to `/login`.

5.2 Status transitions

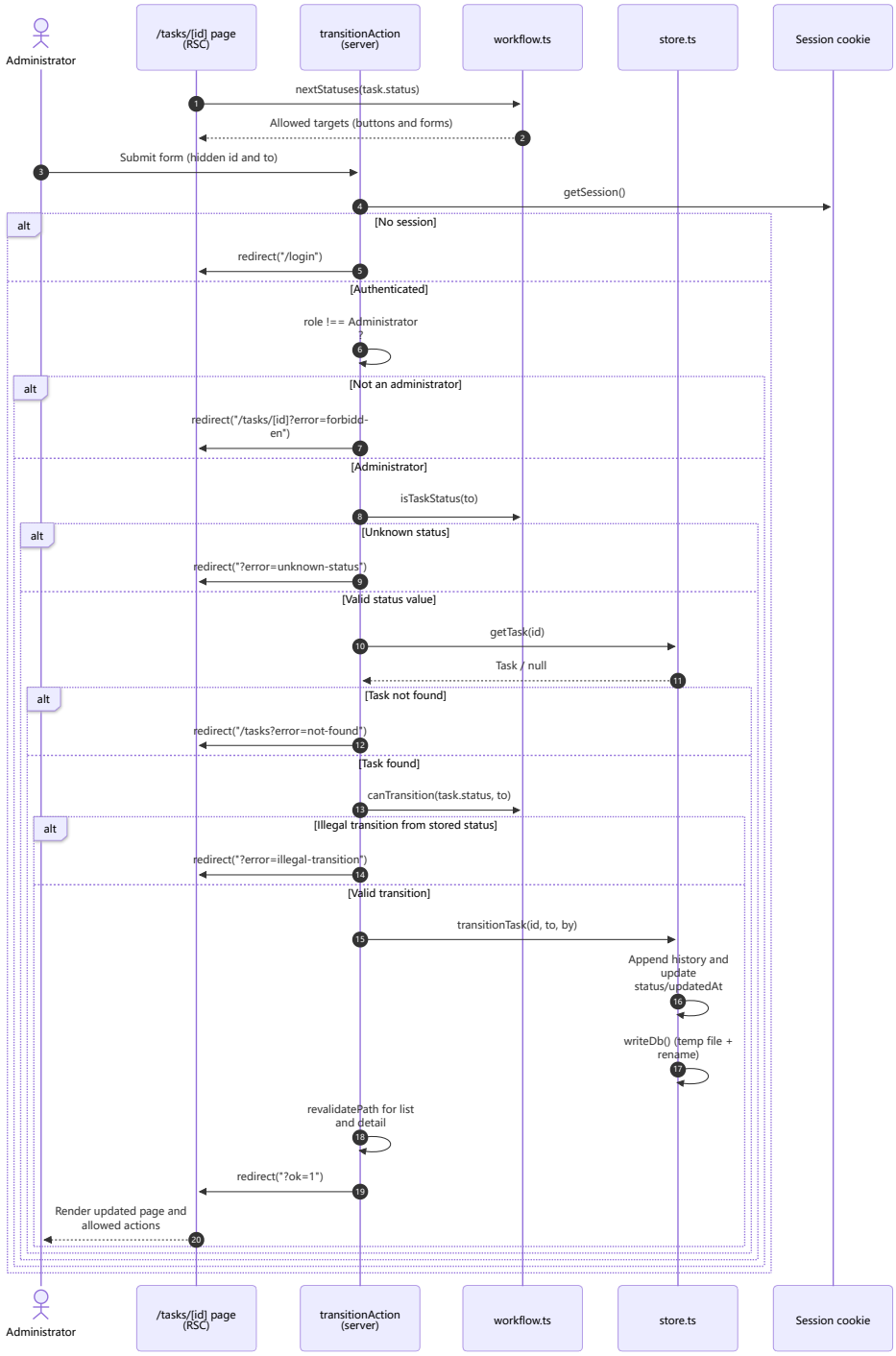


Figure 5. Session and role checks precede status validation, persistence, and revalidation.

Order	Check or action	Failure
1	`getSession()`	Redirect to `/login`
2	Require `session.role === "Administrator"`	`?error=forbidden`

Order	Check or action	Failure
3	`isTaskStatus(to)`	`?error=unknown-status`
4	`getTask(id)`	Redirect to `/tasks?error=not-found`
5	`canTransition(task.status, to)` using the stored current status	`?error=illegal-transition`
6	`transitionTask()` : append history, update status/timestamp, write file	A filesystem failure throws
7	Revalidate list and detail routes, then redirect with `?ok=1`	None

The detail page maps error codes through `ERRORS` and shows a readable message. The list uses `FLASH` for success and error messages.

The action calls `transitionTask(id, to, session.name)` without a note. Runtime events therefore have empty notes; seed history may contain notes. Supporting review comments would require reading a `note` field and passing it to the store.

Transitions read the entire file, modify an in-memory object, and write the entire file back. Concurrent requests can overwrite each other. File failures propagate to Next.js error handling.

5.3 Creation and deletion

Flow	Entry	Validation	Mutation	Result
Create	`/tasks/new` , `NewTaskForm`	Session, Administrator role, title at least 4 characters, description at least 10 characters	Generate ID, set Pending, create initial history event	Redirect to detail
Delete	List or detail delete form	Native browser confirmation; server checks session and Administrator role	Remove task from the array	Redirect to list with deletion message

Input checks currently enforce minimum lengths only. Assignee is unrestricted text. Deleting a nonexistent task still produces a success redirect because the action ignores the boolean returned by `deleteTask()` .

6. Workflow and permissions

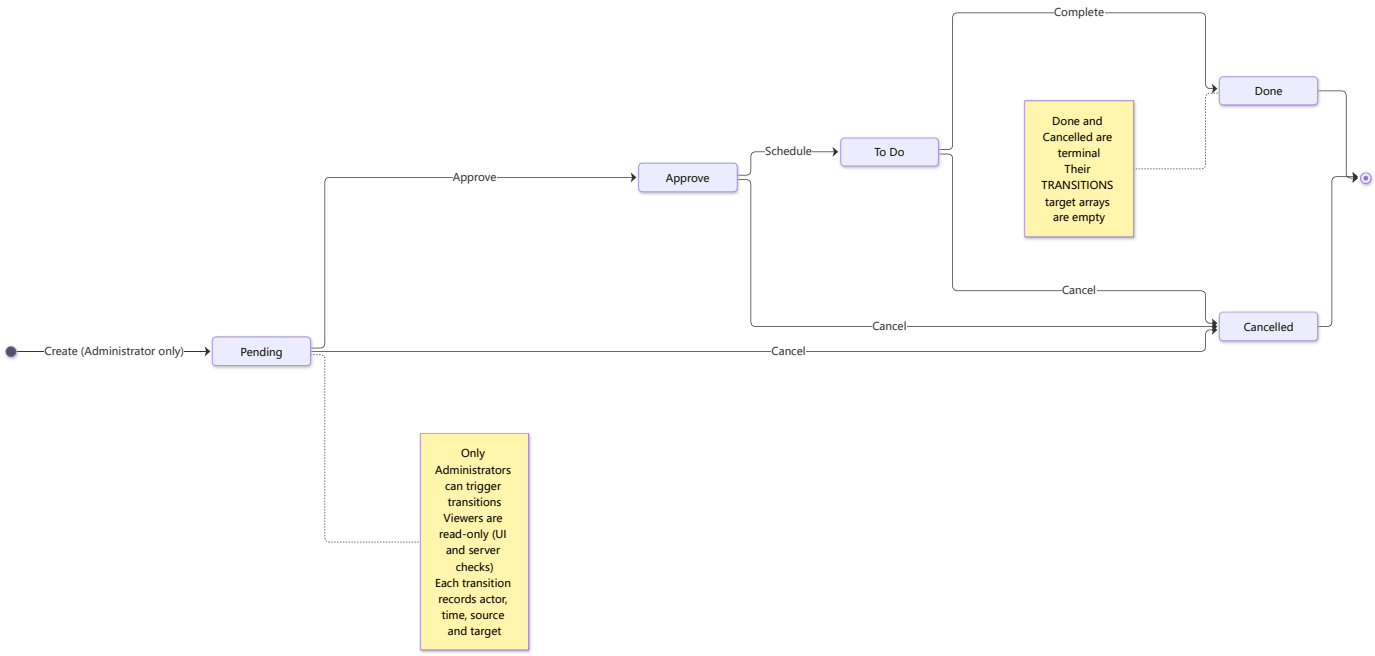


Figure 6. Pending to Approve to To Do to Done; any nonterminal state can become Cancelled. Only administrators trigger transitions.

6.1 States

Status	Meaning	Allowed targets	Terminal
`Pending`	Newly created, awaiting review	`Approve`, `Cancelled`	No
`Approve`	Approved, awaiting scheduling	`To Do`, `Cancelled`	No
`To Do`	Scheduled and in progress	`Done`, `Cancelled`	No
`Done`	Finished	None	Yes
`Cancelled`	Dropped	None	Yes

The single source of truth in `lib/workflow.ts` is:

```
export const TRANSITIONS: Record<TaskStatus, readonly TaskStatus[]> = {
  Pending: ["Approve", "Cancelled"],
  Approve: ["To Do", "Cancelled"],
  "To Do": ["Done", "Cancelled"],
  Done: [],
  Cancelled: [],
};
```

6.2 Role capabilities

Capability	Administrator	Viewer	Enforcement
Read task list and detail	Yes	Yes	`requireSession()`
Read task history	Yes	Yes	Detail page renders history
Create tasks	Yes	No	`requireAdmin()` and `createTaskAction`
Change status	Yes	No	Conditional forms and `transitionAction`
Delete tasks	Yes	No	Conditional forms and `deleteTaskAction`
Visit `/tasks/new`	Yes	Redirect to list	`requireAdmin()`

Viewers are read-only. Allowing them to create tasks would require changing the new-page guard, the action's role check, and the list's create-link condition. Workflow and storage rules could remain unchanged.

6.3 Why permissions are checked twice

```
Page:   isAdmin ? render transition forms : render read-only message
Action: session.role !== "Administrator" -> redirect with error=forbidden
```

The page check prevents confusing or unavailable controls. The server check enforces authorization even if a request is constructed manually.

7. Running, configuring, and testing

7.1 Commands and accounts

```
npm install
npm run dev                 # http://localhost:3000
npm test                    # 13 cases: workflow 8, storage 5
npm run build && npm start  # production mode
```

Email	Password	Role
`admin@example.com`	`admin123`	Administrator
`viewer@example.com`	`viewer123`	Viewer

7.2 Environment variables

Variable	Default	Purpose
`SESSION_SECRET`	Development fallback in source; override for production	Session HMAC key
`DB_PATH`	`<cwd>/data/db.json`	Database file; tests point it to a temporary directory

7.3 Test coverage

File	Cases	Coverage
`test/workflow.test.ts`	8	Initial targets, complete workflow, no skipped or backward transitions, terminal states, all statuses defined, valid targets, forged status rejection
`test/store.test.ts`	5	Seeding, password hashing/verification, Pending creation, status/timestamp/history updates, deletion

Tests configure `DB_PATH` before importing storage, so running them does not modify the real database.

8. Assessment and possible improvements

8.1 Strengths

Strength	Evidence
Centralized workflow rules	`TRANSITIONS` is shared by pages, actions, and tests
Page and server authorization	Role checks in pages and `lib/actions.ts`
Validate before writing	Ordered checks in `transitionAction`
Encapsulated storage	Application filesystem access is confined to `lib/store.ts`
Atomic file replacement	Temporary-file write followed by rename
Small runtime dependency set	Only Next.js, React, and React DOM are direct runtime dependencies
Limited client-side surface	Three Client Components

8.2 Current limitations

Limitation	Effect	Possible improvement
Entire JSON file read and rewritten per mutation	Growing cost and lost concurrent updates	SQLite/Prisma, async store methods, awaited callers
No lock across read-modify-write	Simultaneous transitions can overwrite each other	Version field and conditional updates, or a serialized write queue
Stateless sessions cannot be revoked	A copied token remains valid until expiry even after local sign-out	Server-side session storage or per-user session versions
No explicit CSRF token	Relies on SameSite=Lax and Server Action origin checks	Additional origin validation or tokens for higher-risk operations
Hard deletion	No recovery or deletion history	`deletedAt` and soft-delete filtering

Limitation	Effect	Possible improvement
No pagination or search	Large lists become expensive	Page/query parameters or store query methods
No user administration	Accounts are defined in seed code	Admin user-management and password-reset flows
Development secret fallback	Predictable signing key if production is misconfigured	Fail fast when the production secret is missing
JSON status strings lack database constraints	Manual edits can introduce invalid values	Database enum or CHECK constraint
Any read/parse failure triggers reseeding	Corrupt JSON can silently lose local data	Seed only on ENOENT; otherwise report the error and preserve a backup
Transition notes are not submitted	No runtime review comments	Read and pass a note field
Nonexistent deletion reports success	Misleading feedback	Check `deleteTask()` and report not-found
No maximum input lengths; free-text assignee	Unbounded text and arbitrary assignee values	Maximum lengths and a user selector

8.3 Suggested sequence

Priority	Change	Affected areas
P1	SQLite with Prisma and async store methods	Store and callers in pages/actions
P1	Optimistic locking with a version field	Store and transition action
P2	Soft deletion	Types and storage defaults/filtering
P2	Revocable sessions	Authentication and login action
P3	Pagination, search, CSV export	List page and storage queries
P3	Public task API	A new `app/api/tasks/route.ts` reusing storage and workflow rules

Appendix A. Source file inventory

File	Lines
lib/store.ts	263
app/tasks/[id]/page.tsx	221
app/tasks/page.tsx	194
lib/actions.ts	112
components/LoginForm.tsx	103
lib/auth.ts	93
components/NewTaskForm.tsx	89
test/store.test.ts	80
lib/workflow.ts	76
app/login/page.tsx	72
app/tasks/new/page.tsx	71
test/workflow.test.ts	54
components/TopBar.tsx	52
components/SubmitButton.tsx	45
lib/format.ts	41
lib/types.ts	41

File	Lines
app/layout.tsx	30
app/not-found.tsx	23
lib/password.ts	23
components/StatusBadge.tsx	14
app/page.tsx	7
next.config.ts	7
Total	**1711**

Counts include source lines but exclude a trailing empty line. The local runtime database is not source code. Generated `next-env.d.ts` and non-TS configuration files are excluded from this inventory.

Appendix B. Regenerating diagrams

Diagram sources are in `docs/diagrams/src/`; SVG and PNG outputs are in `docs/images/`.

```
# Install the optional documentation renderer without downloading a browser.
npm install --prefix temp/render-tools --ignore-scripts @mermaid-js/mermaid-cli
node docs/tools/render.mjs
node docs/tools/render.mjs 06-workflow-state.mmd
node docs/tools/render.mjs --svg-only
```

The renderer uses the local Chrome executable configured in `docs/tools/puppeteer-config.json`. Set `MERMAID_CLI_PATH` to use a mermaid-cli installation at another location.

Figure	Source	Content
1. System architecture	`01-system-architecture.mmd`	Three runtime domains, five responsibilities, read/write paths
2. Component tree	`02-component-tree.mmd`	Pages, components, actions
3. Data model	`03-data-model.mmd`	Users, tasks, embedded history
4. Login sequence	`04-login-sequence.mmd`	Login, password verification, session, guards
5. Transition sequence	`05-transition-sequence.mmd`	Validation and persistence
6. Workflow states	`06-workflow-state.mmd`	Five states and role permissions